Logika

МОДУЛЬ 1. УРОК 1

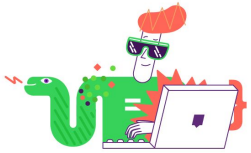
# Урок 1. GIT + Terminal + Venv

Модуль 1. Урок 1

# Вступ

# Основні теми, які будуть вивчатись на занятті:

- Вивчення Git
- Використання терміналу
- Використання віртуальних середовищ (venv)
- Опис SQL
- Django ORM
- Підсумок та завершення



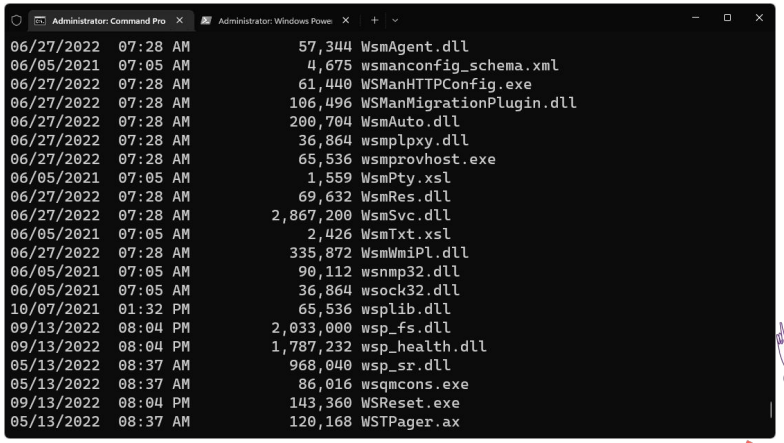
Модуль 1. Урок 1

# Використання терміналу

# Термінал

Термінал - це командний рядок, в якому можна виконувати всі звичні операції на домашньому комп'ютері:

- Вводити різні команди
- Запускати програми
- Аналізувати систему



# Основні команди терміналу

Команда: **ls**

Використовуйте цю команду після переходу до каталогу за допомогою команди cd, щоб переглянути вміст (файли та каталоги) усередині поточного каталогу. Використовуйте аргумент -l (ls -l), щоб отримати ще більше інформації про кожен файл, включаючи дозволи, власника та дату створення.

Приклад: **`ls “path/to/directory/”`**

# Основні команди терміналу

Команда: **cd**

Ця команда змінить каталог, з яким ви зараз працюєте в терміналі, щоб виконувати інші команди в іншому каталозі, переглядати вміст іншого каталогу або відкривати файл в іншому каталозі. Це дуже поширена команда, яка використовуватиметься під час роботи з CLI(command-line interface).

Приклад: **`cd “path/to/directory/”`**

# Основні команди терміналу

Команда: **touch**

Команда touch дозволяє створювати будь-який тип файлу, але він порожній. Створивши порожній файл, ви можете відкрити його в текстовому редакторі, ввівши **`open “filename”`**.

Приклад: **`touch myfile.txt`**

# Основні команди терміналу

Команда: **mkdir**

Ця команда дозволить вам створити каталог (папку) прямо з CLI. Якщо вам потрібно місце для зберігання нових файлів, просто скористайтеся цією командою, щоб додати новий каталог у поточний робочий каталог, або вкажіть повний шлях до розташування, де ви хочете розмістити новий каталог.

Приклад: **`mkdir “path/to/new/directory”`**

# Основні команди терміналу

Команда: **rm**

Ця команда дозволить вам видалити файл в поточній директорії.

Приклад: **`rm my_file.txt`**

Модуль 1. Урок 1

# Вивчення Git

# Що таке Git?

Git — потужна і складна розподільна система контролю версій. Розуміння усіх можливостей git відчиняє для розробника нові горизонти в управлінні програмним кодом.

# Git Flow

Гіт-флоу (Git-flow) - це методологія роботи з версійною системою **Git**, яка надає набір правил та рекомендацій для керування розробкою програмного проекту. Вона використовує специфічну гіт-структуру гілок (**branches**) та набір операцій для організації робочого процесу в команді розробників.

# Git Flow

Основні концепції Git-flow включають в себе:

- Майстер-гілка (**master branch**): Це основна гілка проекту, яка відображає стабільний код та випущені версії.

- Розробницька гілка (**develop branch**): Гілка, що використовується для розробки нових функцій та інтеграції змін в проекті. Вона виходить з майстер-гілки і в неї зливаються гілки функцій (feature branches).

# Git Flow

Основні концепції Git-flow включають в себе:

- Гілки функцій (**feature branches**): Кожна нова функція або зміна розробляється у власній гілці, яка виходить з розробницької гілки. Після завершення розробки вона зливається назад до розробницької гілки.

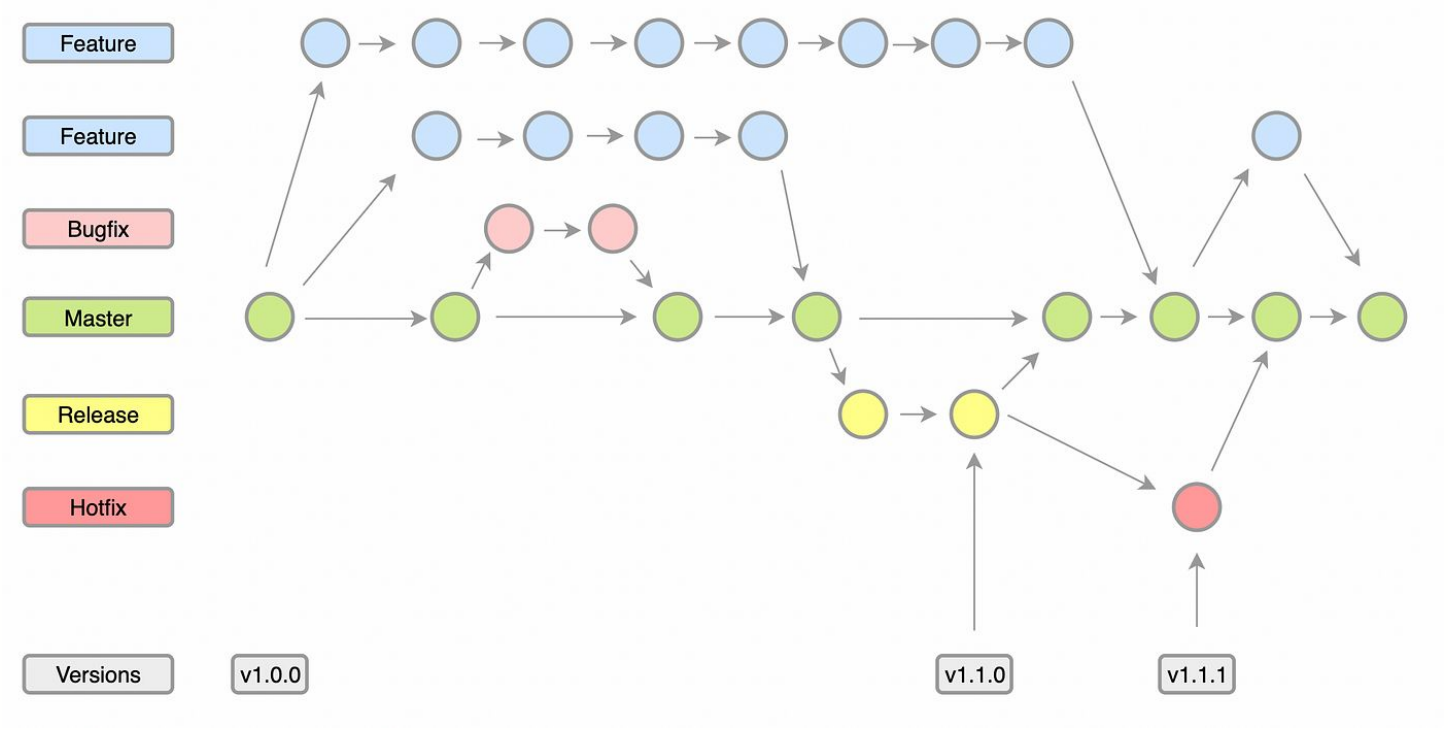
- Гілки випуску (**release branches**): Гілки, що використовуються для підготовки та тестування нових випусків перед об'єднанням з майстер-гілкою.

# Git Flow

Основні концепції Git-flow включають в себе:

- Гілки підтримки (**hotfix branches**): Гілки, які створюються для виправлення критичних помилок випусків, що знаходяться в експлуатації. Вони базуються на майстер-гілці і потім зливаються з майстер-гілкою та розробницькою гілкою

# Git Flow


Feature
Feature
Bugfix
Master
Release
Hotfix
Versions
v1.0.0
v1.1.0
v1.1.1

# Основні команди Git

Команда **git init** використовується для ініціалізації нового репозиторію Git у локальній папці. При виконанні цієї команди Git створює новий підкаталог **.git**, в якому зберігаються всі дані про версії файлів та історію змін.

```
git init
```

# Основні команди Git

Щоб почати відстежувати (додати під версійний контроль) новий файл, використовується команда git add. Щоб розпочати відстеження файлу **README**, ви можете виконати таку команду:

```
git add README
```

# Основні команди Git

Щоб переключитись на певну гілку, використовується команда **checkout**:

```
git checkout main
```

**Checkout** також використовується для створення гілок:

```
git checkout -b feature/my-branch
```

# Основні команди Git

Команда **git commit** зберігає моментальний знімок у журналі проекту та завершує процес відстеження змін. Інакше висловлюючись, фіксація схожа створення фотографії. Все, що було підготовлено за допомогою команди **git add**, стане частиною моментального знімка під час використання **git commit**.

```
git commit -m "мій комміт"
```

# Основні команди Git

Команда **git push** - оновлює віддалений репозиторій з урахуванням фіксацій, виконаних у гілки локально.

```
git push --set-upstream origin my-branch
```

# Основні команди Git

Команда **git pull –** застосовує до локальної лінії розробки оновлення з віддаленого аналога. Розробники використовують цю команду, якщо колега виконав фіксації у галузі віддаленого репозиторію і ці зміни потрібно відобразити в локальному середовищі.

```
git pull
```

# Основні команди Git

Команда **git merge -** виконує злиття ліній розробки. Ця команда зазвичай застосовується для поєднання змін, внесених у двох різних гілках. Наприклад, розробник виконує злиття, коли необхідно поєднати зміни з гілки функції з головною гілкою для розгортання.

```
git merge new-feature
```

Модуль 1. Урок 1

# Використання віртуальних середовищ (venv)

# Віртуальне середовище

**VirtualEnv** використовується для створення віртуальних оточень для програм Python. Це необхідно для того, щоб уникнути конфліктів, дозволяючи встановити одну версію бібліотеки для однієї програми, а іншу — для другої. Вся зручність використання **VirtualEnv** осягається практично.

python™
virtualenv

# Основні причини використання віртуальних середовищ Python:

- **Ізоляція проектів:** Віртуальні середовища дозволяють мати окреме середовище для кожного проекту. Це означає, що ви можете встановити різні версії пакетів для різних проектів і уникнути конфліктів між ними. Ізоляція допомагає зберегти проекти чистими та організованими.

- **Керування залежностями:** Завантаження та управління залежностями проекту може бути складною задачею, особливо якщо ви маєте багато проектів або використовуєте різні версії пакетів. Віртуальні середовища дозволяють легко встановлювати, оновлювати та видаляти пакети без впливу на інші проекти.

# Основні причини використання віртуальних середовищ Python:

- **Робота на різних платформах:** Віртуальні середовища дозволяють створювати переносимі проекти Python, які можна запускати на різних операційних системах. Ви можете створити віртуальне середовище на одній системі та легко перенести його на іншу, забезпечуючи однаковість середовища для вашого проекту.

- **Тестування та експерименти:** Віртуальні середовища дозволяють легко виконувати тести та експерименти з різними версіями пакетів. Ви можете швидко створювати нові середовища для перевірки різних комбінацій залежностей та забезпечення стабільності вашого коду.

# Активація віртуального середовища

Для активації віртуального середовища Python використовуємо наступну команду, залежно від використовуваного інструменту:

Використовуючи **venv** (вбудований в Python 3) та **virtualenv**:

```
source <шлях_до_середовища>/bin/activate
```

На **Windows:**

```
.\<шлях_до_середовища>\Scripts\activate (необхідно запускати в CMD, не в PowerShell)
```

Модуль 1. Урок 1

# Django ORM

# Django

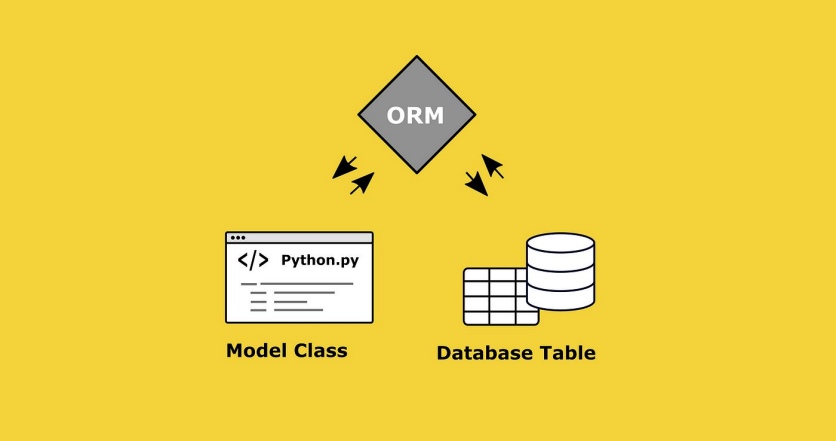
**Django** - це високорівневий веб-фреймворк для розробки веб-додатків на мові програмування Python.

**Основні принципи фреймворку Django:**

- Model-View-Template
- Object Relational Mapping
- Автоматична адміністративна панель
- URL-маршрутизація
- Вбудована підтримка безпеки
- Розширюваність

# Django ORM

**Django ORM (Object-Relational Mapping) -** це інструмент, що надає можливість взаємодіяти з базами даних у розробці веб-додатків за допомогою об'єктно-орієнтованого підходу. Він є частиною фреймворку **Django** і дозволяє розробникам працювати з базами даних, використовуючи **Python**-код, замість безпосереднього написання **SQL**-запитів.



# Важливість Django ORM

Роль **Django ORM** у розробці веб-додатків полягає у спрощенні та стандартизації роботи з базами даних. Ось декілька ключових аспектів, що пояснюють його важливість:

- **Абстракція бази даних:** Django ORM надає абстракцію, яка дозволяє розробникам працювати з базами даних на вищому рівні.
- **Моделі даних:** Django ORM дозволяє описати структуру даних у вигляді моделей Python.
- **Управління залежностями:** Django ORM допомагає керувати залежностями між об'єктами в базі даних.

# Основні функції та можливості Django ORM:

- **Моделі (Models):** Django ORM дозволяє визначати моделі даних, які відображають таблиці бази даних. Клас моделі має поля, які відповідають стовпцям таблиці, а також методи для виконання операцій з даними (створення, зчитування, оновлення, видалення).

- **Міграції (Migrations):** Django ORM надає засоби для автоматичного створення та застосування міграцій. Міграції дозволяють зберігати схему бази даних у вигляді коду та легко змінювати її з часом, безпечно мігруючи дані.

# Основні функції та можливості Django ORM:

- **Запити (Queries):** Django ORM надає потужний API для виконання запитів до бази даних. Ви можете створювати запити з використанням ланцюжків методів (method chaining) або виразів QuerySet для фільтрації, сортування, обмеження результатів і здійснення інших операцій.

- **Відносини (Relationships):** Django ORM підтримує відносини між моделями, такі як один-до-одного, один-до-багатьох і багато-до-багатьох. Ви можете використовувати поля ForeignKey, OneToOneField та ManyToManyField для встановлення зв'язків між моделями.

# Основні функції та можливості Django ORM:

- **Агрегатні функції (Aggregation):** Django ORM дозволяє виконувати агрегатні операції, такі як підрахунок, сумування, середнє значення тощо, на наборах даних.

- **Транзакції (Transactions):** Django ORM підтримує транзакції для забезпечення консистентності бази даних. Ви можете використовувати контекстні менеджери або декоратори, щоб визначити блоки коду, які виконуються в межах транзакції.

# Основні функції та можливості Django ORM:

- **Кешування (Caching):** Django ORM має вбудовану підтримку кешування запитів до бази даних. Ви можете кешувати результати запитів, щоб покращити продуктивність та зменшити навантаження на базу даних.

- **Сигнали (Signals):** Django ORM дозволяє використовувати сигнали для сповіщення про події, пов'язані з моделями. Наприклад, ви можете використовувати сигнали для виконання певних дій перед або після збереження, оновлення або видалення об'єктів моделі.

Модуль 1. Урок 1

# SQL

# Django ORM та SQL

**Django ORM** використовує **SQL** під капотом для взаємодії з базою даних. Однак, **Django ORM** надає вам вищий рівень абстракції, що дозволяє працювати з базою даних без прямого написання складних **SQL**-запитів. Замість цього, ви використовуєте об'єктно-реляційний підхід, де ви визначаєте моделі даних (класи), які відображають таблиці бази даних.

За допомогою **Django ORM**, ви можете створювати, зчитувати, оновлювати та видаляти об'єкти моделі безпосередньо, використовуючи методи і властивості, надані **Django ORM**. Внутрішньо **Django ORM** перетворює ці вищорівневі операції на відповідні **SQL**-запити, які виконуються в базі даних.

Опис SQL

# Основні операції SQL

**SELECT:** Операція **SELECT** використовується для вибірки даних з бази даних. За допомогою **SELECT** можна вибрати конкретні стовпці або всі стовпці з таблиці, використовуючи вирази

```
SELECT * FROM table_name
```

Також можна вказати умови фільтрації з використанням умов **WHERE**.

# Основні операції SQL

**INSERT:** Операція **INSERT** використовується для додавання нових записів у таблицю бази даних. Вона дозволяє вказати назву таблиці та значення для кожного стовпця, які потрібно вставити. Наприклад:

```
INSERT INTO table_name (column1, column2) VALUES (value1, value2)
```

# Основні операції SQL

**UPDATE:** Операція **UPDATE** використовується для оновлення існуючих записів у таблиці бази даних. Вона дозволяє змінювати значення вказаних стовпців відповідно до певних умов. Наприклад:

```
UPDATE table_name SET column1 = value1, column2 = value2
WHERE condition
```

# Основні операції SQL

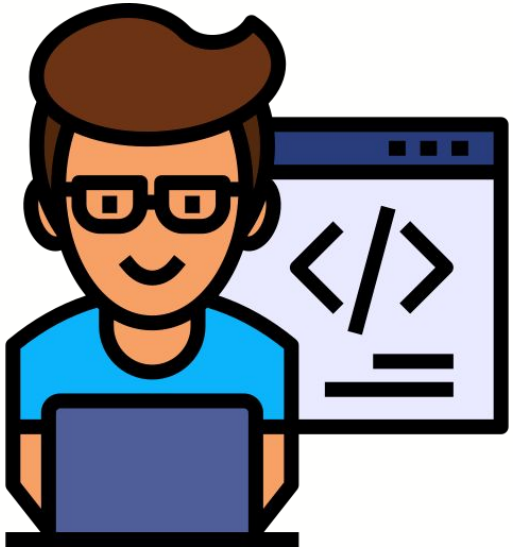
**DELETE:** Операція **DELETE** використовується для видалення записів з таблиці бази даних. Вона дозволяє вказати умови фільтрації для видалення певних записів.
Наприклад:

```
DELETE FROM table_name WHERE condition
```

Модуль 1. Урок 1

# Підсумок

# Теми, які були вивчені на занятті

- Вивчення Git
- Використання терміналу
- Використання віртуальних середовищ (venv)
- Django ORM
- Опис SQL

# Дякуємо за увагу і чекаємо виконаної домашньої практики